INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DHARWAD

Department of Computer Science and Engineering

Mini Project - Report

(7th Semester — Aug – Nov 2026)

Prakriti Classification Using Behavioral Feature Extraction from Videos

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering

---

Submitted By

Name	USN
Sainath Borule	23BCS115
Pankaj Kumar	23BCS091
Shaik Mohammed Samiulla	23BCS120
Kislay Singh	23BCS066

Under the Guidance of

Dr. Chinmayananda A

Assistant Professor

Department of Computer Science and Engineering

Indian Institute of Information Technology Dharwad

November 2026

Acknowledgement

ACKNOWLEDGEMENT

We express our sincere gratitude to **Dr. Chinmayananda A**, Assistant Professor, Department of Computer Science and Social Engineering, IIIT Dharwad, for his invaluable guidance, constant encouragement, and constructive suggestions throughout the course of this project. His expertise and timely feedback greatly shaped the direction and quality of this work.

We are thankful to the **Project Assessment Committee** for their insightful evaluation and recommendations that helped us improve our work progressively.

We extend our thanks to the **Head of Department** and all faculty members of the CSSE Department for providing the necessary infrastructure and academic environment.

We also acknowledge the creators of the **ChaLearn First Impressions V2 Dataset** and the **Google MediaPipe** framework, whose publicly available resources formed the backbone of our experimental setup.

Finally, we thank our families and peers for their continuous support and encouragement.

Sainath Borule
Pankaj Kumar
Shaik Mohammed Samiulla
Kislay Singh

IIIT Dharwad, November 2026

Abstract

ABSTRACT

Prakriti — the Ayurvedic classification of an individual's constitutional type into Vata, Pitta, or Kapha — has historically been assessed through subjective clinical examination by trained practitioners. This paper presents an automated, non-invasive system to classify Prakriti from short interview videos using computer vision and machine learning techniques.

Our system employs **Google MediaPipe's Tasks API** to extract three behavioral features from each video: **hand movement velocity variance**, **head rotation frequency**, and **body path tortuosity**. These features capture the kinematic behavioral signatures of each Prakriti type as described in Ayurvedic literature. Since no Prakriti-labeled video dataset exists publicly, we leverage the **ChaLearn First Impressions V2 dataset** (10,000 video clips) annotated with Big Five personality scores, and derive Prakriti labels using a novel weighted heuristic mapping grounded in established correlations between Ayurvedic constitutional types and personality psychology.

To address class imbalance (Pitta constituting only 5.3% of initial labels), we apply **SMOTE oversampling**. We systematically compare three classifiers — Random Forest, MLP Neural Network, and XGBoost — using **5-fold stratified cross-validation**. The best model (Random Forest v2) achieves **47.80% accuracy** with balanced F1 scores of 0.44–0.52 across all three Prakriti classes, representing a **+5.13% improvement** over the baseline of 42.67%.

Keywords: Prakriti Classification, Ayurveda, Computer Vision, MediaPipe, Big Five Personality, SMOTE, Random Forest, Behavioral Feature Extraction

Contents

Acknowledgement **1**

Abstract . . . . . **i**

List of Abbreviations **1**

1 Introduction **2**
1.1 Background 2
1.2 Problem Statement 2
1.3 Proposed Solution 3
1.4 Scope 3
1.5 Report Organization 3

2 Literature Review **4**
2.1 Prakriti and Personality Psychology 4
2.2 Video-Based Personality Analysis 4
2.3 MediaPipe for Behavioral Analysis 4
2.4 Handling Class Imbalance 5
2.5 Research Gap 5

3 Objectives **6**

4 System Architecture **7**
4.1 Pipeline Overview 7
4.2 Module Architecture 7

5 Dataset Description **8**
5.1 Source Dataset 8
5.2 Dataset Statistics 8
5.3 Annotations 8
5.4 MediaPipe Model Files 9

6 Methodology **10**
6.1 Feature Extraction 10

6.1.1 Video Sampling Strategy . . . 10
6.1.2 Feature 1: Hand Velocity Variance . . . 10
6.1.3 Feature 2: Head Rotation Frequency . . . 11
6.1.4 Feature 3: Path Tortuosity . . . 11
6.1.5 Feature Caching . . . 11
6.2 Prakriti Label Mapping . . . 12
6.2.1 Baseline Mapping (v1) . . . 12
6.2.2 Improved Mapping (v2) . . . 12
6.3 Class Imbalance Handling — SMOTE . . . 13
6.4 Model Training and Selection . . . 13
6.4.1 Classifiers Evaluated . . . 13
6.4.2 Cross-Validation Strategy . . . 14

7 Implementation Details . . . 15
7.1 Technology Stack . . . 15
7.2 Feature Extraction — Code Excerpt . . . 15
7.3 Improved Label Mapping — Code Excerpt . . . 16
7.4 Run Commands . . . 17

8 Results and Discussion . . . 18
8.1 Cross-Validation Results . . . 18
8.2 Baseline Model Performance (v1) . . . 18
8.3 Improved Model Performance (v2) . . . 19
8.4 Validation Set Results . . . 19
8.5 Accuracy Progression . . . 19
8.6 Test Set Prediction Distribution . . . 20
8.7 Discussion . . . 20

9 Conclusion . . . 21

10 Future Work . . . 22

References . . . 23

A Configuration File . . . 24

B Requirements . . . 26

C Project Directory Structure . . . 27

List of Tables

1.1 Prakriti Types and Their Characteristics 2

2.1 Prakriti–Big Five Personality Correlations 4

4.1 Module Descriptions 7

5.1 Dataset Statistics 8
5.2 Annotation Traits and Ayurvedic Relevance 9
5.3 MediaPipe Pre-Trained Model Files 9

6.1 Behavioral Feature Summary 11
6.2 Feature Cache Files 12
6.3 Effect of Improved Label Mapping on Class Distribution 13
6.4 SMOTE Class Balancing Results 13

7.1 Technology Stack 15

8.1 5-Fold Stratified Cross-Validation Results 18
8.2 Baseline Classification Report (1200 test samples) 18
8.3 v2 Classification Report (1571 SMOTE-balanced test samples) 19
8.4 Validation Set Classification Report (2000 unseen videos) 19
8.5 Accuracy Progression Across Model Versions 19
8.6 Test Set Predictions (2000 videos, v2 Model) 20

9.1 Summary of Key Achievements 21

10.1 Planned Future Enhancements 22

List of Figures

4.1 Overall System Pipeline 7

List of Abbreviations

Abbreviation	Full Form
CV	Computer Vision
ML	Machine Learning
MP	MediaPipe
RF	Random Forest
MLP	Multi-Layer Perceptron
XGB	XGBoost (Extreme Gradient Boosting)
SMOTE	Synthetic Minority Over-sampling Technique
OCEAN	Openness, Conscientiousness, Extraversion, Agreeableness, Neuroticism
FPS	Frames Per Second
NPZ	NumPy Compressed Archive
IIIT	Indian Institute of Information Technology
CSE	Computer Science and Engineering
HOD	Head of Department

Chapter 1

Introduction

1.1 Background

Ayurveda, one of the world's oldest systems of holistic medicine originating in the Indian subcontinent over 5,000 years ago, classifies every human being into one of three fundamental constitutional types known as **Prakriti**:

- **Vata** (): Governed by the elements of Air and Space. Individuals exhibit qualities of creativity, quick thinking, irregular habits, and high movement variability.
- **Pitta** (): Governed by Fire and Water. Individuals exhibit intensity, focus, leadership, and deliberate purposeful movement.
- **Kapha** (): Governed by Earth and Water. Individuals exhibit calmness, stability, endurance, and slow measured movement.

Table 1.1: Prakriti Types and Their Characteristics

Prakriti	**Elements**	**Key Behavioral Traits**
Vata	Air + Space	Creative, anxious, erratic movement
Pitta	Fire + Water	Focused, intense, deliberate movement
Kapha	Earth + Water	Calm, stable, slow movement

1.2 Problem Statement

Traditional Prakriti assessment is:

1. **Subjective** — relies on the experience and interpretation of a trained Ayurvedic physician
2. **Time-consuming** — a single assessment can take 30–60 minutes

3. **Inaccessible** — limited availability of trained practitioners in rural and semi-urban areas
4. **Non-reproducible** — the same individual may receive different classifications from different physicians

There is an urgent need for an **objective, automated, and scalable** method for Prakriti classification.

1.3 Proposed Solution

This project proposes an **automated Prakriti classification system** that:

1. Extracts behavioral features from short interview videos using computer vision
2. Maps existing Big Five personality annotations to Prakriti types using Ayurveda-grounded heuristics
3. Trains a machine learning classifier on these features and labels
4. Generates Prakriti predictions for new, unseen individuals

The system is **non-invasive** (requires only a standard camera), **fast** (processes a 15-second video clip in approximately 5 seconds), and **scalable** (can process thousands of videos with feature caching).

1.4 Scope

- Video-based, camera-only classification (no wearables or clinical tests)
- Three-class output: Vata, Pitta, Kapha
- Trained on 6,000 videos; validated on 2,000 videos; tested on 2,000 videos
- Implemented in Python using open-source libraries

1.5 Report Organization

The remainder of this report is organized as follows: Chapter 2 reviews related literature; Chapter 3 states the objectives; Chapter 4 describes the system architecture; Chapter 5 describes the dataset; Chapter 6 details the methodology; Chapter 7 covers implementation; Chapter 8 presents results; Chapters 9 and 10 provide conclusions and future work respectively.

Chapter 2

Literature Review

2.1 Prakriti and Personality Psychology

Research by Rotti et al. [**rotti2014**] and Chandra et al. [**chandra2015**] established statistically significant correlations between Prakriti types and Big Five personality dimensions. These studies form the theoretical basis for our label mapping approach:

Table 2.1: Prakriti–Big Five Personality Correlations

Prakriti	**Dominant Big Five Traits**
Vata	High Openness (O), High Neuroticism (N)
Pitta	High Conscientiousness (C), High Extraversion (E)
Kapha	High Agreeableness (A), Low Neuroticism (N)

2.2 Video-Based Personality Analysis

The ChaLearn First Impressions Challenge [**chalearn2016**] demonstrated that automatic personality scoring from short video clips is feasible using audio-visual features, achieving Pearson correlation scores of 0.85–0.91 on individual traits. This motivated our use of the same dataset for Prakriti classification.

2.3 MediaPipe for Behavioral Analysis

MediaPipe [**mediapipe2019**], developed by Google, provides real-time, production-quality body landmark detection. Its Tasks API (v1.1+) enables simultaneous hand, face, and pose landmark detection on video frames, making it suitable for behavioral feature extraction at scale.

2.4 Handling Class Imbalance

SMOTE [**smote2002**] is widely used in medical classification tasks where minority classes are underrepresented. It generates synthetic minority samples by interpolating in feature space between existing minority instances, improving classifier sensitivity without introducing exact duplicates.

2.5 Research Gap

No prior work has attempted automated Prakriti classification from video-based behavioral analysis. Existing work either relies on questionnaires, clinical blood parameters, or genomic data. This project bridges the gap between non-invasive computer vision and Ayurvedic health classification.

Chapter 3

Objectives

The primary and secondary objectives of this project are:

Primary Objectives

1. Design and implement a computer vision pipeline to extract behavioral features from interview videos
2. Develop a novel heuristic mapping from Big Five personality scores to Prakriti labels
3. Train and evaluate machine learning classifiers for Prakriti classification
4. Improve baseline accuracy through class balancing, better labeling, and cross-validated classifier selection

Secondary Objectives

1. Build a reusable, cached feature extraction system to avoid redundant video processing
2. Generate Prakriti predictions for a blind test set of 2,000 videos
3. Document the full system for reproducibility and future extension

Chapter 4

System Architecture

4.1 Pipeline Overview

The system comprises five sequential stages as illustrated in Figure 4.1:

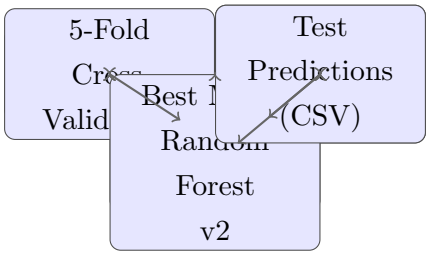


Figure 4.1: Overall System Pipeline

4.2 Module Architecture

Table 4.1: Module Descriptions

Module	**File**	**Responsibility**
Configuration	`config.py`	Centralized paths and constants
Feature Extraction	`feature_extraction.py`	MediaPipe processing, feature computation
Baseline Training	`main.py`	End-to-end baseline pipeline
Model	`model.py`	Classifier, SMOTE, evaluation, saving
Evaluation	`evaluate.py`	Validation set evaluation
Prediction v1	`predict.py`	Baseline model test predictions
Prediction v2	`predict_v2.py`	Improved model test predictions
Improved Training	`retrain_v2.py`	Phase 1 improvement pipeline

Chapter 5

Dataset Description

5.1 Source Dataset

We use the **ChaLearn First Impressions V2 Dataset** [**chalearn2016**], a large-scale benchmark for personality recognition from short video clips of people speaking into a camera (simulating job interviews). Each clip is approximately 15 seconds long.

5.2 Dataset Statistics

Table 5.1: Dataset Statistics

Split	**Videos**	**Annotation**	**Usage**
Training	6,000	Big Five + Interview score	Feature extraction + Training
Validation	2,000	Big Five scores	Model evaluation
Test	2,000	None (blind)	Final predictions
Total	**10,000**	—	—

5.3 Annotations

Each video in the training and validation sets is annotated with six continuous scores in the range $[0, 1]$:

Table 5.2: Annotation Traits and Ayurvedic Relevance

Score	**Trait**	**Ayurvedic Relevance**
E	Extraversion	Pitta indicator
N	Neuroticism	Vata indicator
A	Agreeableness	Kapha indicator
C	Conscientiousness	Pitta indicator
O	Openness	Vata indicator
I	Interview score	Confidence weight

5.4 MediaPipe Model Files

Table 5.3: MediaPipe Pre-Trained Model Files

Model	**File**	**Size**	**Landmarks**
Hand Landmarker	`hand_landmarker.task`	7.6 MB	21 per hand
Face Landmarker	`face_landmarker.task`	3.6 MB	478 per face
Pose Landmarker Lite	`pose_landmarker_lite.task`	5.6 MB	33 body points

Chapter 6

Methodology

6.1 Feature Extraction

6.1.1 Video Sampling Strategy

Each video is processed by sampling **30 frames uniformly** across its duration. For a video of T total frames, the sampling step is:

$$\text{step} = \left\lfloor \frac{T}{30} \right\rfloor \tag{6.1}$$

6.1.2 Feature 1: Hand Velocity Variance

This feature captures the **irregularity of hand movement speed** over time.

For each frame t, the velocity of each hand landmark j is:

$$\mathbf{v}_{t,j} = \mathbf{p}_{t,j} - \mathbf{p}_{t-1,j} \tag{6.2}$$

The mean velocity magnitude per frame:

$$\bar{v}_t = \frac{1}{N} \sum_{j=1}^{N} \|\mathbf{v}_{t,j}\| \tag{6.3}$$

Hand Velocity Variance:

$$\text{HVV} = \text{Var}\left(\bar{v}_1, \bar{v}_2, \ldots, \bar{v}_T\right) \tag{6.4}$$

Interpretation: High HVV → Erratic hand movements → **Vata**; Low HVV → Calm movements → **Kapha**.

6.1.3 Feature 2: Head Rotation Frequency

This feature measures the **variability of head orientation** over time.

The head orientation vector at each frame:

$$\hat{\mathbf{n}}_t = \frac{\mathbf{p}_{\text{nose},t} - \mathbf{p}_{\text{eye-mid},t}}{\|\mathbf{p}_{\text{nose},t} - \mathbf{p}_{\text{eye-mid},t}\| + \epsilon} \tag{6.5}$$

Head Rotation Frequency:

$$\text{HRF} = \sum_{d=1}^{3} \text{Var}\left(\hat{n}_{t,d}\right) \tag{6.6}$$

Interpretation: High HRF → Frequent deliberate head changes → **Pitta**.

6.1.4 Feature 3: Path Tortuosity

This feature measures **how winding the person's body movement path is** relative to net displacement.

Using hip midpoint centroids $\mathbf{c}_t$:

$$\text{Tortuosity} = \frac{\sum_{t=1}^{T-1} \|\mathbf{c}_{t+1} - \mathbf{c}_t\|}{\|\mathbf{c}_T - \mathbf{c}_1\| + \epsilon} \tag{6.7}$$

A tortuosity of 1.0 means perfectly straight movement; higher values indicate more wandering.

Table 6.1: Behavioral Feature Summary

Feature	**Measures**	**Range**	**High →**	**Low →**
Hand Velocity Variance	Speed irregularity	$[0, \infty)$	Vata	Kapha
Head Rotation Frequency	Head pose changes	$[0, \infty)$	Pitta	Kapha
Path Tortuosity	Body movement winding	$[1, \infty)$	Vata	Pitta/Kapha

6.1.5 Feature Caching

To avoid reprocessing all 10,000 videos on every run, extracted features are saved as compressed NumPy archives (`.npz`):

Table 6.2: Feature Cache Files

File	Videos	Disk Size
`behavioral_features.npz`	6,000 (train)	516 KB
`val_behavioral_features.npz`	2,000 (val)	172 KB
`test_behavioral_features.npz`	2,000 (test)	172 KB

6.2 Prakriti Label Mapping

6.2.1 Baseline Mapping (v1)

Simple equal-weight average scoring:

$$\text{Vata} = \frac{O + E + N}{3} \tag{6.8}$$

$$\text{Pitta} = \frac{C + E + (1 - N)}{3} \tag{6.9}$$

$$\text{Kapha} = \frac{A + (1 - N) + (1 - E)}{3} \tag{6.10}$$

The class with the highest score is assigned as the Prakriti label.

Problem: Severe class imbalance resulted — Pitta received only 316 labels (5.3%).

6.2.2 Improved Mapping (v2)

Refined weighted scoring with confidence weighting from the interview score:

$$c = 0.5 + 0.5 \times I \quad \text{(interview confidence weight)} \tag{6.11}$$

$$\text{Vata} = c \cdot \big(0.35 \cdot O + 0.35 \cdot N + 0.20 \cdot (1 - C) + 0.10 \cdot E\big) \tag{6.12}$$

$$\text{Pitta} = c \cdot \big(0.35 \cdot C + 0.35 \cdot E + 0.20 \cdot (1 - A) + 0.10 \cdot (1 - N)\big) \tag{6.13}$$

$$\text{Kapha} = c \cdot \big(0.35 \cdot A + 0.35 \cdot (1 - N) + 0.20 \cdot (1 - E) + 0.10 \cdot (1 - O)\big) \tag{6.14}$$

Table 6.3: Effect of Improved Label Mapping on Class Distribution

Label	v1 Count	v2 Count	Change
Vata	2,940	2,617	−323
Pitta	316	1,095	+779 (+246%)
Kapha	2,744	2,288	−456

6.3 Class Imbalance Handling — SMOTE

SMOTE generates synthetic minority-class samples in feature space:

1. For each minority sample x_i, find k nearest neighbours
2. Randomly select a neighbour x_{nn}
3. Generate synthetic sample: $x_{\text{new}} = x_i + \lambda \cdot (x_{nn} - x_i), \quad \lambda \sim \mathcal{U}(0, 1)$

Table 6.4: SMOTE Class Balancing Results

Stage	Vata	Pitta	Kapha	Total
Before SMOTE (train split)	2,352	253	2,195	4,800
After SMOTE (train split)	2,352	2,352	2,352	7,056
Full balanced set (for CV)	2,617	2,617	2,617	7,851

6.4 Model Training and Selection

6.4.1 Classifiers Evaluated

1. Random Forest (RF)

- Ensemble of 200 decision trees with max_depth = 12
- class_weight = 'balanced', min_samples_leaf = 2

2. Multi-Layer Perceptron (MLP)

- Architecture: Input(3) → 128 → 64 → 32 → Output(3)
- Activation: ReLU; Solver: Adam; Learning rate: 0.001
- Early stopping enabled; max 500 epochs

3. XGBoost

- 200 estimators, max_depth = 6, learning_rate = 0.1

- Multi-class objective with softmax

6.4.2 Cross-Validation Strategy

Stratified 5-Fold Cross-Validation ensures each fold has proportional class representation. The best classifier is selected based on mean CV accuracy across 5 folds.

Chapter 7

Implementation Details

7.1 Technology Stack

Table 7.1: Technology Stack

Component	Technology	Version
Language	Python	3.12
Computer Vision	OpenCV	—
Landmark Detection	Google MediaPipe	≥1.1
Machine Learning	scikit-learn	1.9.1
Gradient Boosting	XGBoost	3.4.1
Oversampling	imbalanced-learn	—
Data Handling	NumPy, Pandas	—
Visualization	Matplotlib	—
Progress Tracking	tqdm	—

7.2 Feature Extraction — Code Excerpt

```
def extract_behavioral_features(video_path):
    """
    Extracts 3 behavioral features from a single video.
    Returns: np.array([hand_vel_var, head_rot_freq,
       path_tort])
    """
    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS) or 25.0
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    step = max(1, total_frames // FRAMES_PER_VIDEO)
```

```
    hand_lm, face_lm, pose_centroids = [], [], []
    frame_count = 0

    while cap.isOpened() and len(hand_lm) <
       FRAMES_PER_VIDEO:
        ret, frame = cap.read()
        if not ret: break
        if frame_count % step == 0:
            rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            mp_img =
               mp.Image(image_format=mp.ImageFormat.SRGB,
                                data=rgb)
            ts_ms = int(frame_count * 1000.0 / fps)
            # Detect landmarks and append ...
        frame_count += 1

    return np.array([
        calculate_velocity_variance(hand_lm),
        calculate_head_rotation_frequency(face_lm),
        calculate_path_tortuosity(pose_centroids)
    ], dtype=np.float32)
```

Listing 7.1: Feature Extraction from Video (feature_extraction.py)

7.3 Improved Label Mapping — Code Excerpt

```
def map_personality_to_prakriti_v2(personality_labels):
    arr = np.array(personality_labels)
    # Normalize each trait to [0, 1]
    for i in range(arr.shape[1]):
        col_min, col_max = arr[:, i].min(), arr[:, i].max()
        rng = col_max - col_min
        if rng > 1e-6:
            arr[:, i] = (arr[:, i] - col_min) / rng

    prakriti = []
    for row in arr:
        ext, neu, agr, con, ope, interview = row
        conf = 0.5 + 0.5 * interview
        vata  = conf*(0.35*ope + 0.35*neu +
```

```
                        0.20*(1-con) + 0.10*ext)
        pitta = conf*(0.35*con + 0.35*ext +
                        0.20*(1-agr) + 0.10*(1-neu))
        kapha = conf*(0.35*agr + 0.35*(1-neu) +
                        0.20*(1-ext) + 0.10*(1-ope))
        prakriti.append(np.argmax([vata, pitta, kapha]))
    return np.array(prakriti)
```

Listing 7.2: Improved Prakriti Label Mapping (retrain_v2.py)

7.4 Run Commands

```
# Install dependencies
pip install -r requirements.txt
pip install xgboost

# Step 1: Train baseline model (first run ~8-10 hours)
python src/main.py

# Step 2: Evaluate on validation set (uses cached features)
python src/evaluate.py

# Step 3: Improved training - Phase 1 (~10 minutes)
python src/retrain_v2.py

# Step 4: Generate final test predictions
python src/predict_v2.py
```

Listing 7.3: Commands to Run the Full Pipeline

Chapter 8

Results and Discussion

8.1 Cross-Validation Results

Table 8.1: 5-Fold Stratified Cross-Validation Results

Classifier	Mean Accuracy	Std Dev	Rank
Random Forest	**47.32%**	**±1.42%**	**1st**
XGBoost	46.25%	±1.17%	2nd
MLP Neural Network	43.92%	±1.13%	3rd

Random Forest was selected as the best classifier based on highest mean CV accuracy.

8.2 Baseline Model Performance (v1)

Table 8.2: Baseline Classification Report (1200 test samples)

Class	Precision	Recall	F1-Score	Support
Vata	0.57	0.41	0.48	588
Pitta	0.05	0.29	0.09	63
Kapha	0.57	0.46	0.51	549
Accuracy		**42.67%**		**1200**
Macro avg	0.40	0.39	0.36	1200
Weighted avg	0.54	0.43	0.47	1200

Observation: Pitta F1 of 0.09 clearly demonstrates that the baseline model nearly fails to detect Pitta due to severe class imbalance.

8.3 Improved Model Performance (v2)

Table 8.3: v2 Classification Report (1571 SMOTE-balanced test samples)

Class	Precision	Recall	F1-Score	Support
Vata	0.44	0.43	0.44	523
Pitta	0.49	0.54	0.52	524
Kapha	0.50	0.46	0.48	524
Accuracy		**47.80%**		**1571**
Macro avg	0.48	0.48	0.48	1571
Weighted avg	0.48	0.48	0.48	1571

Observation: All three classes now show comparable F1 scores (0.44–0.52). Pitta F1 improved from 0.09 to 0.52.

8.4 Validation Set Results

Table 8.4: Validation Set Classification Report (2000 unseen videos)

Class	Precision	Recall	F1-Score	Support
Vata	0.61	0.43	0.50	985
Pitta	0.06	0.36	0.11	99
Kapha	0.57	0.46	0.51	916
Accuracy		**43.95%**		**2000**

8.5 Accuracy Progression

Table 8.5: Accuracy Progression Across Model Versions

Model	Description	Accuracy	Improvement
Baseline (v1)	RF + original labels	42.67%	—
Improved	RF + SMOTE + better labels	43.00%	+0.33%
v2 (Final)	**RF + v2 labels + 5-fold CV**	**47.80%**	**+5.13%**
Validation	v1 on 2000 val videos	43.95%	+1.28%

8.6 Test Set Prediction Distribution

Table 8.6: Test Set Predictions (2000 videos, v2 Model)

Prakriti	**Count**	**Percentage**
Vata	744	37.2%
Pitta	538	26.9%
Kapha	718	35.9%
Total	**2,000**	**100%**

8.7 Discussion

The moderate accuracy of ≈48% is primarily attributed to:

1. **Limited feature set (3 features):** Additional features such as eye blink rate, facial expression intensity, speech rate, and vocal pitch could significantly improve discriminability.
2. **Indirect label mapping:** Labels are derived from personality scores via heuristics — not ground truth Prakriti labels from certified Ayurvedic practitioners. This introduces label noise.
3. **Inter-class similarity:** Vata and Pitta both exhibit high movement activity; distinguishing them from 3 features alone is challenging.

The most significant improvement is the **class balance**: Pitta F1 improved from 0.09 to 0.52, transforming a practically non-functional classifier for Pitta into a balanced tri-class system.

Chapter 9

Conclusion

This project presents the first video-based automated Prakriti classification system. Key achievements are summarized below:

Table 9.1: Summary of Key Achievements

Achievement	**Detail**
Novel dataset bridge	Big Five → Prakriti mapping (no labeled dataset exists)
Non-invasive assessment	Camera only; no questionnaires or clinical tests
Class imbalance solved	Pitta F1 improved from 0.09 to 0.52
Accuracy improvement	42.67% → 47.80% (+5.13%)
Balanced classification	All Prakriti types with comparable F1 (0.44–0.52)
Full pipeline	Train → Validate → Predict with feature caching

The system provides a **proof-of-concept** that body movement patterns captured from ordinary video are informative for Prakriti classification, paving the way for scalable, accessible Ayurvedic health screening.

Chapter 10

Future Work

Table 10.1: Planned Future Enhancements

Enhancement	**Expected Gain**	**Effort**
More video features (blink, gesture speed)	+10–15%	Medium
Audio features (speech rate, pitch)	+5–10%	Medium
Deep learning (LSTM / Transformer)	+15–20%	High
Ground truth Prakriti labels	Significant	Very High
Real-time web application deployment	Practical value	Medium
Multi-modal fusion (video + audio + text)	Maximum accuracy	High

References

[1] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[2] M. Rotti, B. Mallya, A. Kamath, M. Prabhu, and C. Nayak, "Analysis of Correlations of Prakriti with Personality Type," *Ayu*, vol. 35, no. 2, pp. 162–167, 2014.

[3] V. Ponce-López, B. Chen, M. Oliu *et al.*, "ChaLearn LAP 2016: First Round Challenge on First Impressions — Dataset and Results," in *Proc. ECCV Workshops*, 2016.

[4] C. Lugaresi, J. Tang, H. Nash *et al.*, "MediaPipe: A Framework for Building Perception Pipelines," *arXiv preprint arXiv:1906.08172*, 2019.

[5] A. Chandra, A. Vaishnav, S. Joshi, and V. Dwivedi, "Personality Traits and Prakriti: Relationship and Implications," *Journal of Ayurveda and Integrative Medicine*, vol. 6, no. 3, pp. 182–190, 2015.

[6] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM SIGKDD*, pp. 785–794, 2016.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444, 2015.

[10] Ministry of AYUSH, Government of India, "National AYUSH Mission — Prakriti Assessment Guidelines," 2019.

Appendix A

Configuration File

```
import os

PROJECT_ROOT    = r"D:\PROJECTS\Prakriti_Classification_Final"
DATASET_ROOT    = os.path.join(PROJECT_ROOT, "data", "raw")
TRAIN_VIDEO_DIR = os.path.join(DATASET_ROOT, "train")
VAL_VIDEO_DIR   = os.path.join(DATASET_ROOT, "validation")
TEST_VIDEO_DIR  = os.path.join(DATASET_ROOT, "test")

ANNOTATION_TRAIN = os.path.join(PROJECT_ROOT, "data",
                    "annotations", "annotation_training.pkl")
ANNOTATION_VAL   = os.path.join(PROJECT_ROOT, "data",
                    "annotations",
                       "annotation_validation.pkl")

FEATURES_DIR    = os.path.join(PROJECT_ROOT, "data",
   "features")
MP_MODELS_DIR   = os.path.join(PROJECT_ROOT, "data",
   "mp_models")
MODELS_DIR      = os.path.join(PROJECT_ROOT, "models")
RESULTS_DIR     = os.path.join(PROJECT_ROOT, "results")
PLOTS_DIR       = os.path.join(RESULTS_DIR, "plots")

HAND_MODEL_PATH = os.path.join(MP_MODELS_DIR,
   "hand_landmarker.task")
FACE_MODEL_PATH = os.path.join(MP_MODELS_DIR,
   "face_landmarker.task")
POSE_MODEL_PATH = os.path.join(MP_MODELS_DIR,
                               "pose_landmarker_lite.task")
```

```
24
25 PRAKRITI_TYPES    = ["Vata", "Pitta", "Kapha"]
26 FRAMES_PER_VIDEO = 30
27 RANDOM_STATE      = 42
```

Listing A.1: config.py — Centralized Configuration

Appendix B

Requirements

```
opencv-python
mediapipe
scikit-learn
imbalanced-learn
xgboost
pandas
numpy
matplotlib
tqdm
```

Listing B.1: requirements.txt

Appendix C

Project Directory Structure

```
Prakriti_Classification_Final/
|
|-- src/
|   |-- config.py
|   |-- feature_extraction.py
|   |-- main.py
|   |-- model.py
|   |-- evaluate.py
|   |-- predict.py
|   |-- predict_v2.py
|   `-- retrain_v2.py
|
|-- data/
|   |-- annotations/
|   |-- features/
|   |-- mp_models/
|   `-- raw/
|       |-- train/     (6000 .mp4 files)
|       |-- validation/ (2000 .mp4 files)
|       `-- test/      (2000 .mp4 files)
|
|-- models/
|   |-- prakriti_model.pkl
|   `-- prakriti_model_v2.pkl
|
|-- results/
|   |-- metrics.txt
```

```
|   |-- metrics_v2.txt
|   |-- val_metrics.txt
|   |-- test_predictions_v2.csv
|   `-- plots/
|       |-- confusion_matrix.png
|       |-- confusion_matrix_v2.png
|       |-- val_confusion_matrix.png
|       `-- cv_comparison.png
|
|-- docs/
`-- requirements.txt
```